Universidade Federal de Mato Grosso

Disciplina: Estrutura de Dados 1 Docente: Dr. Ivairton

Discente: Humberto Henrique Castro Rocha Matrícula: 202512722003

Discente: Paulo Henrique Gonçalves Coelho Matrícula: 202511722024

Relatório — *04 de dezembro de 2025*

Análise comparativa de *lista estática* x *lista dinâmica*

Introdução

O presente relatório tem como objetivo apresentar a análise de desempenho computacional entre três estruturas de dados fundamentais: *Lista Estática* (vetor) , *Lista Dinâmica Simplesmente Encadeada* e *Lista Dinâmica Duplamente Encadeada*. Conforme proposto no Trabalho, foi desenvolvido um sistema para inserir 10.000 registros inteiros aleatórios, seguindo uma distribuição específica de inserções (início, fim e posições aleatórias), medindo-se o tempo de processamento e analisando o consumo de memória.

Metodologia

Estratégia de Desenvolvimento e Divisão de Tarefas

O desenvolvimento do sistema foi realizado de forma colaborativa e modularizada, visando a organização do código e a facilidade de manutenção. O projeto foi segmentado em bibliotecas (arquivos `.h` e `.c`), separando as implementações das estruturas de dados da lógica principal de execução. A divisão de responsabilidades foi estabelecida da seguinte forma:

- **Humberto Henrique:** Responsável pela implementação completa da **Lista Estática** (bibliotecas e funções de manipulação de vetor) e pela elaboração deste

relatório técnico e análise dos dados.

- **Paulo Henrique:** Responsável pela implementação das **Listas Dinâmicas** (simplesmente e duplamente encadeadas) e pelo desenvolvimento do arquivo **main.c**, que orquestra os testes e gera as saídas no terminal.

0.1 Algoritmo e Medição de Tempo

O sistema foi desenvolvido na linguagem `C`. Para a medição do tempo de processamento, utilizou-se a biblioteca `sys/times.h`, conforme solicitado. O algoritmo realiza os seguintes passos para cada estrutura:

- $\approx 1/3$ (3.333 registros) inseridos no **início**.
- $\approx 1/3$ (3.333 registros) inseridos no **fim**.
- $\approx 1/3$ (3.334 registros) inseridos em **posições aleatórias**.

Assim como exigido, o experimento foi repetido 10 vezes para garantir a consistência estatística dos dados (média, menor e maior tempo).

Resultados Obtidos

Abaixo são apresentados os resultados das 10 execuções realizadas. Os valores de tempo estão em segundos.

Tabela 1: Comparativo de Desempenho (Inserção de 10.000 dados)

Estrutura	**Qtd. Ins. Início**	**Qtd. Ins. Fim**	**Qtd. Ins. Posição**	**Menor tempo**	**Maior tempo**	**Tempo médio**
Lista estática (vetor)	3333	3333	3334	**0.01320**	**0.02300**	**0.01650**
Lista dinâmica (simples)	3333	3333	3334	**0.05280**	**0.06980**	**0.05834**
Lista dinâmica (dupla)	3333	3333	3334	**0.06050**	**0.08720**	**0.06960**

O tempo de processador foi medido utilizando a biblioteca `‘sys/times.h‘`, garantindo precisão na coleta do tempo de CPU utilizado pelo processo. Foram realizadas 10

execuções completas para cada estrutura, permitindo o cálculo de métricas estatísticas (mínimo, máximo e média) e amenizando variações pontuais do SO.

Análise e Discussão

Análise de Tempo (Custo Computacional)

Observou-se que a **Lista Estática** obteve o melhor desempenho (menor tempo). Isso ocorre devido à alocação adjacente de memória, que favorece o uso do cache do processador, e à ausência da sobrecarga de alocação dinâmica (`malloc`) para cada nó individual.
As listas dinâmicas apresentaram tempos superiores. Logo, a **Lista Duplamente Encadeada** foi a mais lenta, pois, além da alocação de memória para cada nó, exige a manipulação de dois ponteiros (próximo e anterior) a cada inserção, aumentando o número de operações elementares.

Análise de Memória

Em relação à ocupação de memória para armazenar 10.000 inteiros (considerando `int` = 4 bytes e ponteiro = 8 bytes em arquitetura 64-bits):

- **Lista Estática:** Ocupa apenas o espaço dos dados.
  10.000×4 bytes $= 40.000$ bytes ≈ 40 KB.
- **Lista Dinâmica Simples:** Ocupa dados + 1 ponteiro por nó.
  $10.000 \times (4 + 8)$ bytes $= 120.000$ bytes ≈ 120 KB.
- **Lista Dinâmica Dupla:** Ocupa dados + 2 ponteiros por nó.
  $10.000 \times (4 + 8 + 8)$ bytes $= 200.000$ bytes ≈ 200 KB.

Portanto, a Lista Estática é o método mais eficiente em relação ao uso de memória, utilizando cerca de 3 a 5 vezes menos memória que as versões dinâmicas neste cenário.

Conclusão

Conclui-se que, para um cenário onde o limite de dados é conhecido (10.000) e a velocidade é crítica, a Lista Estática é superior tanto em tempo quanto em economia de memória. As listas dinâmicas, apesar de flexíveis quanto ao tamanho, introduzem "custos indiretos"significativos de processamento e memória.

Código Fonte Abaixo apenas para visualização

Abaixo apresenta-se a implementação do sistema, dividida em módulos para melhor organização e manutenção do código. O desenvolvimento foi segmentado em bibliotecas para as listas estáticas e dinâmicas, além do arquivo principal.

Arquivo: main.c

Listing 1: Função principal e orquestração dos testes

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include "estatica.h"
#include "dinamica.h"

#define NUM_ITERACOES 10
#define TAMANHO 10000

// serve para calcular o tempo em segundos
double calcularTempo(clock_t inicio, clock_t fim) {
    return ((double)(fim - inicio)) / CLOCKS_PER_SEC;
}

// -- Oque acontece em um teste tambem acontece no outro, o tempo calculado, a logica de inserção --
// -- Todo o código que faz a inserção fica separado, depois são unidos na hora da compilação     --
// -- Esse foi o jeito q eu o Beto fizemos para dividir as tarefas, ficou mais complexo mas valeu --

void realizarTesteDesempenho() {
    clock_t t_inicio, t_fim;
    double tempo_acumulado_estatica = 0;
    double tempo_acumulado_simples = 0;
    double tempo_acumulado_dupla = 0;
    int i, k, valor, pos;

    srand(time(NULL));
    // ------------------------------------------------------------
    // |      TESTE LISTA ESTATICA                                |
    // ------------------------------------------------------------
    for (k = 0; k < NUM_ITERACOES; k++) {
        ListaEstatica lista;
```

```
32          inicializaLista(&lista);
33
34          t_inicio = clock(); //salva o tempo de inicio
35
36          for (i = 0; i < TAMANHO; i++) {
37              valor = rand() % 10000;
38
39              // primeiro ele coloca no inicio, depois no meio e depois no
                    fim da lista
40
41              if (i < 3333) {
42                  insereInicioEstatica(&lista, valor); //chama a funcao
                        que esta lá em lista_estatica.c
43              } else if (i < 6666) {
44                  insereFimEstatica(&lista, valor); //chama a funcao que
                        esta lá em lista_estatica.c
45              } else {
46                  pos = rand() % (i + 1);
47                  inserePosicaoEstatica(&lista, valor, pos); //chama a
                        funcao que esta lá em lista_estatica.c -> a MESMA
                        coisa pros outros teste, oq muda é que vai chamar a
                        funcao que esta em listadinamica.c
48              }
49          }
50          t_fim = clock(); //salva o tempo de fim
51          tempo_acumulado_estatica += calcularTempo(t_inicio, t_fim); //
                calcula o tempo gasto
52      }
53
54      // ------------------------------------------------------------
55      // |       TESTE DINAMICA SIMPLES                             |
56      // ------------------------------------------------------------
57      for (k = 0; k < NUM_ITERACOES; k++) {
58          no *listaSimples = NULL;
59
60          t_inicio = clock();
61
```

```
62          for (i = 0; i < TAMANHO; i++) {
63              valor = rand() % 10000;
64
65              if (i < 3333) {
66                  inserirSimples(&listaSimples, valor, 0, 0);
67              } else if (i < 6666) {
68                  inserirSimples(&listaSimples, valor, i, 0);
69              } else {
70                  pos = rand() % (i + 1);
71                  inserirSimples(&listaSimples, valor, pos, 0);
72              }
73          }
74          t_fim = clock();
75          tempo_acumulado_simples += calcularTempo(t_inicio, t_fim);
76
77          removerSimples(listaSimples);
78      }
79
80      // ------------------------------------------------------------
81      // |   TESTE DINAMICA DUPLA                                   |
82      // ------------------------------------------------------------
83      for (k = 0; k < NUM_ITERACOES; k++) {
84          noDuplo *listaDupla = NULL;
85
86          t_inicio = clock();
87
88          for (i = 0; i < TAMANHO; i++) {
89              valor = rand() % 10000;
90
91              if (i < 3333) {
92                  inserirDupla(&listaDupla, valor, 0);
93              } else if (i < 6666) {
94                  inserirDupla(&listaDupla, valor, i);
95              } else {
96                  pos = rand() % (i + 1);
97                  inserirDupla(&listaDupla, valor, pos);
98              }
```

```
99          }
100         t_fim = clock();
101         tempo_acumulado_dupla += calcularTempo(t_inicio, t_fim);
102
103         removerDupla(listaDupla);
104     }
105
106     printf("media do tempo estatica: %f segundos\n",
            tempo_acumulado_estatica / NUM_ITERACOES);
107     printf("media do tempo dinamica simples: %f segundos\n",
            tempo_acumulado_simples / NUM_ITERACOES);
108     printf("media do tempo dinamica dupla %f segundos\n",
            tempo_acumulado_dupla / NUM_ITERACOES);
109 }
110
111 int main() {
112     realizarTesteDesempenho();
113     return 0;
114 }
```

Arquivo: estatica.h

Listing 2: Função principal e orquestração dos testes

```
1
2  // Cabeçalho para lista estática
3  #ifndef ESTATICA_H
4  #define ESTATICA_H
5
6  //Feito pelo HH
7
8  #include <stdio.h>
9  #include <stdlib.h>
10
11 #define MAX 10000
12
13 typedef struct {
```

```
14      int dados[MAX];
15      int quantidade;
16  } ListaEstatica;
17
18  // Prototipos das funcoes
19  void inicializaLista (ListaEstatica *lst);
20  void insereInicioEstatica (ListaEstatica *lst, int valor);
21  void insereFimEstatica (ListaEstatica *lst, int valor);
22  void inserePosicaoEstatica (ListaEstatica *lst, int valor, int pos);
23  int removeInicioEstatica (ListaEstatica *lst);
24  int removeFimEstatica (ListaEstatica *lst);
25  int removePosicaoEstatica (ListaEstatica *lst, int pos);
26
27  #endif
```

Arquivo: lista_estatica.c

Listing 3: Implementação da Lista Estática

```
1
2   #include "estatica.h"
3   #include<stdlib.h>
4   #include<stdio.h>
5   //Feito pelo HH
6
7   //Implementacao das funcoes da lista ESTATICA:
8
9   //Cria uma LISTA estatica vazia
10  void inicializaLista (ListaEstatica *lst) {
11      lst->quantidade = 0;
12  }
13
14  //Insere um elemento no INICIO da LISTA estatica
15  void insereInicioEstatica (ListaEstatica *lst , int valor) {
16      if (lst->quantidade >= MAX) {
17          printf("[ERRO] Lista cheia! nao eh possivel inserir novo
                elemento.\n");
```

```
18          return;
19      } else {
20          int i;
21          for (i = lst->quantidade; i > 0; i--) {
22              lst->dados[i] = lst->dados[i - 1];
23          }
24          lst->dados[0] = valor;
25          lst->quantidade++;
26      }
27
28  }
29
30  //Insere um elemento no FIM da LISTA estatica
31  void insereFimEstatica (ListaEstatica *lst, int valor) {
32      if (lst->quantidade >= MAX) {
33          printf("[ERRO] Lista cheia! nao eh possivel inserir novo
                elemento.\n");
34          return;
35      } else {
36          if (lst->quantidade < MAX) {
37              lst->dados[lst->quantidade++] = valor;
38          }
39      }
40  }
41
42  //INSERE um elemento em uma POSICAO especifica da LISTA
43  void inserePosicaoEstatica (ListaEstatica *lst, int valor, int pos){
44      if (lst->quantidade >= MAX) {
45          printf("[ERRO] Lista cheia! nao eh possivel inserir novo
                elemento.\n");
46          return;
47      } else {
48          if ((pos >= 0) && (pos <= lst->quantidade)) {
49              int i;
50              for(i = lst->quantidade-1; i >= pos; i--) {
51                  lst->dados[i+1] = lst->dados[i];
52              }
```

```
53              lst->dados[i+1] = valor;
54              lst->quantidade++;
55          }
56
57          }
58
59  }
60
61  //REMOVE um elemento do INICIO da LISTA estatica
62  int removeInicioEstatica (ListaEstatica *lst) {
63      if (lst->quantidade == 0) {
64          printf("[ERRO] Lista vazia! nao eh possivel remover elemento.\n"
                );
65          return -1; // Valor de erro
66      } else {
67          int valorRemovido, i;
68          valorRemovido = lst->dados[0];
69          //desloca os elementos para a esquerda
70          for (i = 0; i < lst->quantidade - 1; i++) {
71              lst->dados[i] = lst->dados[i + 1];
72          }
73          lst->quantidade--;
74          return valorRemovido;
75      }
76  }
77
78  //REMOVE um elemento do FIM da LISTA estatica
79  int removeFimEstatica (ListaEstatica *lst) {
80      if (lst->quantidade == 0) {
81          printf("[ERRO] Lista vazia! nao eh possivel remover elemento.\n"
                );
82          return -1; // Valor de erro
83      } else {
84          return lst->dados[--lst->quantidade];
85      }
86  }
87
```

```c
//REMOVE um elemento de uma POSICAO especifica da LISTA estatica
int removePosicaoEstatica (ListaEstatica *lst, int pos) {
    if (lst->quantidade == 0) {
        printf("[ERRO] Lista vazia! nao eh possivel remover elemento.\n");
        return -1; // Valor de erro
    } else {
        if ((pos < 0) || (pos >= lst->quantidade)) {
            printf("[ERRO] Posicao invalida! nao eh possivel remover elemento.\n");
            return -1; // Valor de erro
        } else {
            int valorRemovido, i;
            valorRemovido = lst->dados[pos];
            //desloca os elementos para a esquerda
            for (i = pos; i < lst->quantidade - 1; i++) {
                lst->dados[i] = lst->dados[i + 1];
            }
            lst->quantidade--;
            return valorRemovido;
        }
    }
}
```

Arquivo: dinamica.h

Listing 4: Definições das Listas Dinâmicas

```c

#ifndef DINAMICA_H
#define DINAMICA_H

//Feito pelo PH

#include<stdlib.h>
#include<stdio.h>

```

```
10 //------estrutura comum------//
11 typedef struct no{
12     int valor;
13     struct no *prox;
14 }no;
15
16 //------estrutura dupla------//
17 typedef struct noDuplo{
18     int valor;
19     struct noDuplo *noProx;
20     struct noDuplo *noAnt;
21 }noDuplo;
22
23 //------prototipo das funções------//
24 void inserirSimples(no **lp, int valor, int posicao, int total);
25 void removerSimples(no *lp);
26
27 void inserirDupla(noDuplo **lp, int valor, int posicao);
28 void removerDupla(noDuplo *lp);
29
30
31
32 #endif
```

Arquivo: lista_dinamica.c

Listing 5: Implementação das Listas Dinâmicas

```
1
2 #include<stdlib.h>
3 #include<stdio.h>
4 #include "dinamica.h"
5
6 //Feito pelo PH
7
8 //------Inserção na lista dinamica simples------//
9
```

```
10 void inserirSimples(no **lp, int valor, int posicao, int total){
11     no *novo =  (no*)malloc(sizeof(no));
12     novo->valor = valor;
13     novo->prox = NULL;
14 
15     if (posicao == 0){
16         novo->prox = *lp;
17         *lp = novo;
18         return; // serve para retornar direto, como ja encontramos a
               posição que queriamos, não precisamos continuar na funcao
19     }
20 
21     no *atual = *lp;
22     int i = 0;
23     while (atual->prox != NULL && i < posicao - 1){
24         atual = atual->prox;
25         i++;
26     }
27 
28     novo->prox = atual->prox;
29     atual->prox = novo;
30 
31 }
32 
33 //------Inserção na lista dinamica dupla------//
34 
35 void inserirDupla(noDuplo **lp, int valor, int posicao){
36     noDuplo *novo = (noDuplo*)malloc(sizeof(noDuplo));
37     novo->valor = valor;
38     novo->noProx = NULL;
39     novo->noAnt = NULL;
40 
41     if (posicao == 0){
42         novo->noProx = *lp;
43         if (*lp != NULL){
44             (*lp)->noAnt = novo;
45 
```

```
46          }
47      *lp = novo;
48      return;
49      }
50
51      noDuplo *atual = *lp;
52      int i = 0;
53
54      while(atual->noProx != NULL && i < posicao - 1){
55          atual = atual->noProx;
56          i++;
57      }
58
59      novo->noProx = atual->noProx;
60      novo->noAnt = atual;
61
62      if(atual->noProx != NULL){
63          atual->noProx->noAnt = novo;
64      }
65      atual->noProx = novo;
66  }
67
68  //------Remoção da lista simples------//
69
70  void removerSimples(no *lp) {
71      no *atual = lp;
72      while (atual != NULL) {
73          no *temporario = atual;
74          atual = atual->prox;
75          free(temporario);
76      }
77  }
78
79  //------Remoção da lista Dupla------//
80
81  void removerDupla(noDuplo *lp) {
82      noDuplo *atual = lp;
```

```
83      while (atual != NULL) {
84          noDuplo *temporario = atual;
85          atual = atual->noProx;
86          free(temporario);
87      }
88  }
```